

Upcraft Educational Video System

Governing design policy & implementation notes

Compiled documentation set · includes video generation process,
benchmarks, model routing, react-to-MP4 rendering analysis, and
source-grounded lesson planning

Upcraft Educational Video System Documentation

These documents are the product's governing design policy. Read them in this order before changing the pipeline:

1. [Video generation process](#) — the required workflow, dependencies, and reasons for each stage.
2. [Benchmarks to focus on](#) — the quality scorecard, release gates, and domain policies.
3. [Model recommendations](#) — approved model routing, fallbacks, and cost rationale.

The repository-level [agent instructions](#) make these policies mandatory. Any user-requested change that conflicts with them requires explicit confirmation before implementation, followed by synchronized updates to every affected document and test.

Additional implementation note:

- [React-to-MP4 rendering and deterministic spatial compositing](#) — reference-repository analysis and a non-governing design for headless rendering, audio/caption alignment, and constraint-based image layering.
- [Source-grounded lesson planning and structured scene manifests](#) — implementation-backed walkthrough of what the planning job creates and forwards, with a worked photosynthesis example.

All documentation and code in this repository are proprietary to Upcraft Solutions Private Limited. See [LICENSE](#), [CONTRIBUTING](#), and [AUTHORS](#).

Educational Video Generation Process

This process creates a continuous whiteboard/motion-graphics lesson, not a slideshow. Remotion owns deterministic typography, diagrams, animation, captions, and timing. Image models create only illustrative assets that benefit from generative art.

Models propose bounded artifacts; typed validation, source-grounded verification, deterministic rendering, and release gates decide whether an artifact may advance. A weak, unavailable, or malformed model response must never silently become a lower-quality released lesson.

Change control

This is a governing document under [AGENTS.md](#). Any request to alter a required stage, its ordering, parallelism rules, reliability controls, or release behavior must be explained and explicitly confirmed with the question tool when available before implementation. Update the benchmark and model-recommendation documents in the same approved change whenever this workflow changes their requirements.

Reliability contract

Every generation is one durable video run. This workflow deliberately does not define classroom, collaboration, chat-session, multi-user, or browser-session behavior.

Before any billable or irreversible work, the system must reserve a unique intake-session or run/output identity, freeze an immutable input snapshot, validate required provider/model/storage/renderer/approval capabilities, and persist a versioned manifest in `queued` state. The snapshot contains intake settings, supplied sources, requested destination, and selected capability configuration. Chat requests reserve an `intake_sessions` record before the Intake Briefing Agent is called; the later video run is created only from the validated brief.

Each stage consumes only locked upstream artifacts and produces a versioned typed artifact. Its checkpoint records input and output hashes, schema version, prompt/template and model/provider versions, attempt count, validation evidence, source links where applicable, cost, latency, timestamps, and an explicit outcome. Generated bytes additionally require a content hash, MIME type, size, and provenance.

Provider context is also versioned. The research stage may receive the complete immutable source snapshot once and must emit a lossless source-evidence map. Later model calls receive only the smallest deterministic projection of locked artifacts needed by that stage plus the exact source segments referenced by the claims under review. Projection metadata records artifact hashes, segment IDs, item counts, and character counts; it must never replace evidence with an unverified model summary or silently truncate source text. Provider usage records input, cached-input, output, and reasoning tokens where available, modality-specific units such as TTS characters, request identifiers, pricing-version metadata, cost, latency, and outcome without storing duplicate private prompts.

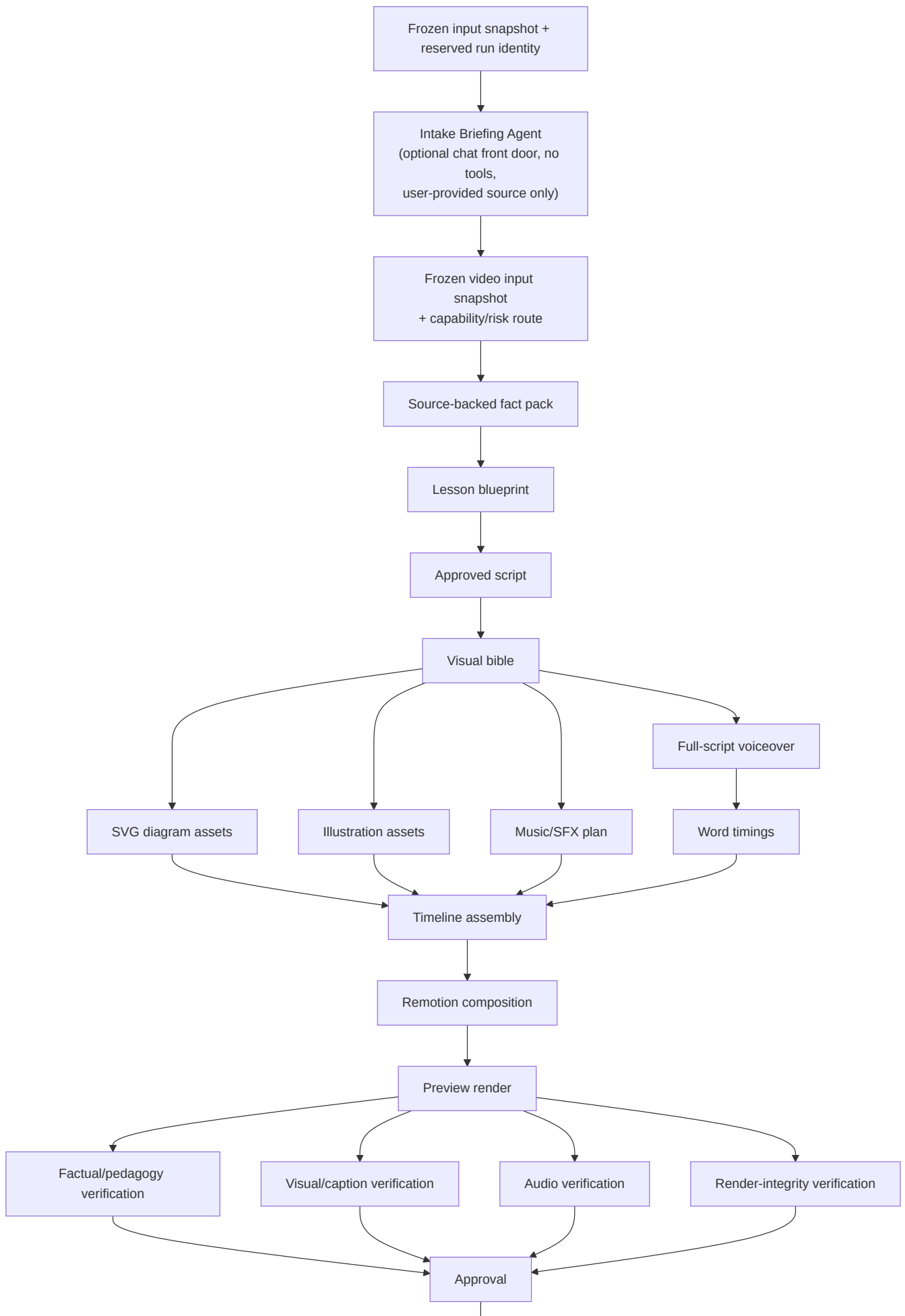
The approved script stores narration lines as the single canonical text representation. Full narration text for TTS is derived deterministically from those lines, so a downstream stage cannot create a second divergent copy of the script.

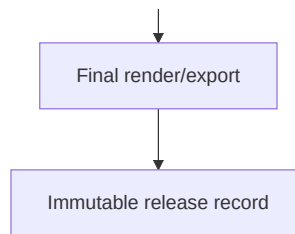
Run status is explicit: `queued`, `running`, `awaiting_approval`, `failed`, or `completed`. A restart or expired lease must recover from the latest valid checkpoint or leave the run visibly failed; it must never appear successful. Replaying a stage with the same locked inputs must use stable artifact identities and idempotent writes, so it cannot duplicate assets or overwrite a newer valid artifact.

Validation, retry, and fallback rules

- Treat model text, tool output, external URLs, generated media, and provider metadata as untrusted input.
- Validate every generated artifact against its stage schema before marking it valid. Deterministic normalization may repair transport-only defects, such as a fenced JSON wrapper; it must not invent facts, labels, narration, timings, or visual meaning.
- Retry only bounded, classified transient failures—rate limits, timeouts, temporary provider errors, and network errors—with exponential backoff and jitter. Persist every attempt. Do not retry cancellation, authentication/configuration errors, quota exhaustion, malformed inputs, or failed safety/release gates as though they were transient.
- An invalid model artifact may be regenerated as a new bounded attempt with the validation error, but it is never promoted by syntactic repair alone.
- Fact packs, blueprints, approved scripts, voice tracks, timings, project manifests, preview renders, QA reports, and final renders are critical artifacts: no valid artifact means the run is blocked or failed.
- Decorative illustrations and optional music/SFX may fall back only to an approved existing asset, deterministic Remotion/SVG treatment, or omission. Record the reason and choice. Factual diagrams, labels, equations, charts, and captions must always use typed SVG/Remotion components.
- A valid partial result is not a completed video. The release record must name every required artifact and every approved optional fallback.

Dependency map





Parallel work begins only after its inputs are locked. A scene may not generate images before the visual bible is approved, and Remotion may not finalize timing before the voice track is available. A failed downstream artifact may be regenerated without reopening a locked upstream artifact only when its input contract remains valid.

Sub-processes

1. Chat intake briefing — sequential, optional front door

The Chat toggle accepts one request, a language selector, and one user-provided source: TXT/Markdown/PDF upload, an authoritative URL, or a `Source:` text block. The durable Intake Briefing Agent (`intake-briefing-agent`) receives only the request and selected language, uses GPT-5.6 Luna through the AI Gateway, has no tools, and returns the typed `intake-brief/v1` object: topic, learner level, risk domain, audience, duration, language, and visual profile. It must not research, cite, write narration, create assets, or invent source material. The service stores the request hash, brief hash, model/prompt version, usage, latency, and every bounded attempt in `intake_sessions` / `intake_attempts` . A valid brief is then passed to the normal run creator with the supplied source; a failed or unavailable briefing never starts video generation.

Source files are extracted deterministically (UTF-8 TXT/Markdown or text-based PDF), size- and character-limited, hashed, and stored privately before the queued session is dispatched. The chat front door does not perform web search. URL retrieval and source-grounded research remain later pipeline responsibilities and must still pass their existing gates.

2. Intake and risk routing — sequential

Collect topic, target learner level, language, duration, aspect ratio, brand/style profile, and destination from the validated brief or the advanced form. Classify the domain as standard, engineering, medical, or client-production. Freeze this input, reserve the run/output identity, and fail early if required providers, credentials, storage, render capacity, or approval paths are unavailable.

Why: a primary-school photosynthesis lesson and a medical-school pharmacology lesson cannot share the same source, safety, or review policy.

2. Research and fact pack — sequential

Retrieve authoritative, topic-appropriate sources. Extract only the facts, definitions, calculations, caveats, and citations required for the stated learning objective. Record material claim-to-source links, stable source locators or snapshots, retrieval time, and source suitability. Independently verify every material claim before planning.

Why: the script must be written from evidence, rather than asking a model to remember science or medicine from training data.

3. Lesson blueprint — sequential

Create the measurable learning objective, learner prerequisites, hook, explanation arc, recap, and optional knowledge-check. Divide the explanation into scenes and visual beats; each beat introduces one idea and one meaningful canvas change. Validate the typed blueprint for learning-objective coverage, prerequisites, scene order, and claim references before locking it.

Why: this is where the system becomes a teacher rather than a generic story generator.

4. Script and storyboard approval — sequential

Write spoken narration from the fact pack, then attach each line to a scene purpose, on-screen text, visual action, and source claims. Validate factual claims, reading level, duration, and safety policy before the script becomes immutable. A separate verifier or deterministic claim check must evaluate it against the fact pack; the script-writing model cannot be the only authority.

Why: changing the script after audio, captions, and scene assets exist causes needless cost and sync errors.

5. Visual bible — sequential

Create one project-level specification: canvas texture, marker/line style, palette, typography, character/object sheets, icon rules, camera behavior, caption design, and prohibited visual patterns. Define persistent entities such as the same leaf, sun, molecule, circuit, or patient diagram. Lock the visual-bible version with the script and attach it to every downstream asset brief.

Why: it prevents visual drift and lets scenes feel like one evolving whiteboard.

6. Asset production — parallel by scene after the visual bible locks

- **SVG/Remotion diagrams:** Build molecules, arrows, charts, labels, equations, and annotated processes as editable vector components.
- **Illustrations:** Generate only scene illustrations, characters, or textures; provide the visual bible and relevant reference assets with every prompt. Verify returned media type, dimensions, bytes, content hash, provenance, and prompt/style adherence before attaching it to the run.
- **Sound plan:** Select minimal music and sound effects, leaving narration intelligible. Treat this as optional enrichment with an explicit fallback or omission record.

Each candidate asset has a stable identity derived from the run, scene, role, and attempt. Multiple candidates may be evaluated in parallel, but only a validated selected asset may become a scene reference.

Why: these outputs have no dependency on one another once scene direction is fixed. Vectors protect factual accuracy; illustrations add warmth and context.

7. Voiceover and captions — sequential, then derivation

Generate one voiceover for the complete approved narration with pronunciation notes. Validate audio bytes, duration, loudness, voice identity, and curated-domain-term pronunciation. Obtain word/character alignment from the TTS response, derive phrase captions, and reserve short pauses for diagram inspection. Check captions against the locked script; do not independently paraphrase them or treat guessed timing as equivalent to rendered narration.

Why: one voice track preserves tone, pacing, and transitions. Captions must come from the rendered narration, not an independently paraphrased script.

8. Timeline assembly and composition – sequential

Convert scene durations, audio timings, captions, camera moves, selected asset identifiers, and transitions into a typed versioned project manifest. Remotion renders the sequence as a persistent canvas: draw/reveal paths, retain established objects, pan/zoom to the next concept, and use explicit transitions only where the teaching context changes. The composition must consume verified manifest references, never unverified provider URLs; persist the renderer, composition, dependency, and export-profile versions required to reproduce a render.

Why: a manifest makes renders reproducible and lets the renderer remain deterministic even when upstream AI output varies.

9. Quality assurance – parallel checks after preview render

Render a preview from the locked manifest, then run independent checks concurrently:

- schema, artifact-completeness, source-claim, and policy validation;
- factual/pedagogy review against the fact pack and learning objective;
- visual continuity, diagram semantics, safe-area, caption-overlap, and generated-text inspection;
- audio duration, pronunciation, word-timing, loudness, and music-ducking checks;
- render integrity, asset availability, and export-profile tests.

Every finding must identify the failed artifact, rule, evidence, and remediation. Regenerate or replace only the earliest failed downstream artifact whose inputs remain locked, then rerun dependent checks. A validator must not mark its own generated artifact verified without an independent source, deterministic test, or separately routed verifier.

Why: each validator sees a different failure class; a single model cannot reliably judge its own output.

10. Approval, render, and feedback loop – sequential

Require the correct approval level, then render the MP4 and optional SRT/transcript from the approved manifest. The immutable release record must preserve the input snapshot, source pack and claim links, prompt/template and model/provider versions, asset provenance and hashes, attempt/retry history, manifest, QA evidence, approval, cost, latency, and final-output hashes. A failed, stale, blocked, or incomplete run must remain visibly non-publishable. Feed viewer retention, quiz results, and reviewer feedback into the regression suite.

Why: reproducibility and measured learning outcomes are necessary to improve the system safely.

Required parallelism rules

May run in parallel	Must wait for
Per-scene SVG assets, illustrations, and sound planning	Approved script and visual bible
Independent factual, pedagogy, visual, and audio QA	Preview render and its manifest
Multiple candidates for one approved optional illustration	Scene asset brief, references, and candidate identity
Multiple render resolutions/formats	Approved master composition

Must remain sequential	Reason
Chat briefing -> frozen video input snapshot/capability route -> research -> lesson blueprint -> script approval	Prevents mixed inputs and unsupported narration; the briefing cannot become a hidden research step.
Script approval -> complete voiceover -> caption timing	Prevents visual/audio/caption drift.
Visual bible -> persistent asset generation	Preserves scene continuity.
Asset/timing completion -> final timeline -> final render	Keeps the composition deterministic.
QA approval -> publication	Prevents known errors from reaching learners.

On failure, resume from the latest validated checkpoint only after rechecking that its locked inputs, dependencies, and provider capabilities remain valid. Never parallelize a retry with an active attempt for the same artifact identity, and never overwrite a validated artifact without recording a new version and reason for replacement.

Benchmarks to Focus On

This is the release-quality scorecard for the educational-video pipeline. A general model leaderboard is only a **candidate-selection signal**; a video may ship only when it passes the task-specific checks below. Update this document whenever `video-generation-process.md` adds, removes, or changes a stage.

Change control

This is a governing document under `AGENTS.md`. Before changing a benchmark, release gate, threshold, or domain policy, explain the impact and obtain explicit user confirmation through the question tool when available. In the same approved change, update the generation process, model recommendations, regression suite, and implementation wherever they are affected.

Candidate model signals

Capability	External signal	How we use it
General reasoning, STEM, instruction following	Artificial Analysis Intelligence Index	Shortlist text models; compare quality, latency, and price.
Human preference for explanations	LMArena	Tie-breaker for voice, clarity, and writing quality; never a factuality gate.
Health safety and communication	HealthBench	Required signal for medical-model candidates, alongside our clinician review set.
Image quality and value	Artificial Analysis Image Arena	Shortlist visual models; validate continuity on our own storyboards.
Voice quality	Internal listening and pronunciation suite	Select TTS by naturalness, pronunciation, timing, and language support; there is no sufficient universal public TTS leaderboard.

Release benchmark suite

Maintain a versioned, teacher-reviewed evaluation set split into school science, college STEM, engineering, medical education, and production explainers. Every example includes authoritative sources, expected learning objective, expected claims, visual requirements, and prohibited errors.

Pipeline stage	What to measure	Initial release gate	Failure action
Chat intake briefing	Structured fields from short requests, conservative medical routing, selected-language preservation, no invented sources/claims, and valid user-source requirement	100% schema validity and routing regression; 0 fabricated source/citation/narration fields; 100% of accepted sessions have a TXT/Markdown/PDF, URL, or pasted source	Hold the session; fix prompt/schema/route or require a source.

Pipeline stage	What to measure	Initial release gate	Failure action
Intake durability and provenance	Session reservation before Luna call, input/brief hashes, attempt records, bounded retries, private upload hashes, and handoff to exactly one video run	100% of required records present; duplicate delivery creates no second run; no raw private source in telemetry	Block intake release and repair persistence/queue/idempotency.
Input routing	Correct audience, domain, risk level, language, and format selection	100% on the routing regression set	Fix routing policy or require clarification.
Run integrity	Immutable input snapshot, capability preflight, versioned artifacts, checkpoint records, stable artifact identities, and complete provenance	100% of required records present; 0 unclassified terminal states	Block the run; repair orchestration or persistence before retrying.
Usage and context accounting	Every provider attempt has request metadata, token/unit usage, projection version, artifact/segment references, pricing version, cost, latency, and outcome; telemetry contains no raw private prompt/source duplication	100% of provider attempts accounted for; no missing usage outcome	Keep the run auditable; repair provider accounting before accepting cost comparisons.
Recovery and retries	Retry only classified transient failures; bounded attempts; idempotent replay from the latest valid checkpoint	100% on retry, stale-run, duplicate-write, and resume regression cases	Block release; fix retry classification, checkpointing, or idempotency.
Research	Every material claim has a direct, authoritative source; sources are current enough for the topic	100% cited material claims; no unsupported high-stakes claim	Reject the lesson plan and research again.
Lesson plan	Clear learning objective, prerequisite handling, age-appropriate language, and one teachable idea per visual beat	Teacher score $\geq 4/5$ average; no critical pedagogy issue	Revise plan before scripting.
Script accuracy	Correct facts, calculations, units, terminology, uncertainty, and safety language	100% on critical claims; $\geq 95\%$ overall supported claims	Regenerate only the failed scene/script section.
Medical safety	Accuracy, appropriate uncertainty, non-diagnostic language, red-flag escalation, and source quality	Clinician approval is mandatory	Never auto-publish; hold for clinician revision.

Pipeline stage	What to measure	Initial release gate	Failure action
Visual plan	Each narration beat has a useful visual action; persistent objects retain identity; diagrams are semantically correct	0 misleading diagrams, illegible labels, or continuity breaks	Replace the scene asset/animation plan.
Image assets	Prompt adherence, recurring-character/object consistency, style adherence, and absence of invented scientific text	$\geq 4/5$ reviewer score; 0 factual labels embedded in generated images	Regenerate, edit, or replace with SVG.
Captions	Exact wording, readable line length, speaker synchronization, contrast, and safe-area placement	0 overflow/overlap defects; word alignment $p95 \leq 150$ ms	Rebuild caption timings/layout.
Voiceover	Pronunciation of domain terms, natural pacing, no clipping, and stable voice identity	$\geq 4/5$ listener score; 100% of curated terms accepted	Add pronunciation notes or regenerate audio.
Render	Valid duration, audio/video sync, asset loading, no dropped frames, and correct export profile	100% automated smoke tests pass	Block render and fix the composition.
Release record	Reproducible final output with locked manifest, approval, QA evidence, hashes, source links, provider/model versions, and cost/latency data	100% required release-record fields present; no unresolved critical QA result	Keep the output non-publishable and rebuild the missing or failed artifact.
Viewer outcome	Intro retention, scene drops, rewatches, quiz performance, and teacher feedback	Track by video and audience segment	Turn weak scenes into new regression examples.

Domain policy

- School and college lessons may publish automatically only after automated source, script, visual, caption, and render gates pass.
- Engineering lessons must show units, assumptions, and calculation steps; a wrong dimension, scale, or safety instruction is critical.
- Medical lessons are educational, not diagnostic or prescriptive. They require current clinical sources and qualified human approval before release.
- Production-company work additionally requires client style approval and a rights/provenance record for every third-party asset.

Evaluation cadence

Run the small regression suite on every prompt, model, composition, schema, retry, checkpoint, context-projection, or workflow-contract change. Include chat requests such as “photosynthesis working”, explicit field overrides, conservative medical routing, missing/invalid source, TXT/Markdown/PDF extraction, URL-only input, malformed Luna structured output, transient and permanent provider failures, stale-run recovery, duplicate execution, missing required assets, optional-asset fallback, source-verification failure, stale/missing evidence segments, canonical narration reconstruction, private-telemetry leakage, and blocked medical publication. Run the full set before changing a default model or promoting a new context projection. Store model/version, prompt version, source pack, projection manifest, output, token/unit usage, cost, latency, per-gate result, retry/checkpoint evidence, and reviewer feedback so a later model or prompt change is comparable rather than subjective. Establish the first cost baseline from 20 accepted videos; subsequent comparable runs over 125% of the cohort p75 are alert-and-review events, not automatic publication blockers.

Model Recommendations by Job

Model choices are a snapshot as of 2026-09-17. Providers and scores change quickly; use `benchmarkstofocus.md` before replacing a default.

Change control

This is a governing document under `AGENTS.md`. Replacing a default model, fallback, or role boundary requires explicit user confirmation through the question tool when available. The approved change must include the candidate benchmark evidence, cost/latency comparison, any workflow impact, and updates to the regression suite.

Production defaults

Job	Default	Why	Fallback / escalation
Intake Briefing Agent (chat metadata extraction)	OpenAI GPT-5.6 Luna via AI Gateway	The user-approved low-cost route is sufficient for strict structured metadata and conservative risk classification. The agent is toolless and is not a factual, source, narration, or release authority.	Hold intake if the route is unavailable or malformed; do not silently substitute a model without a new policy decision.
Source-grounded lesson planning and structured scene manifests	OpenAI GPT-5.6 Terra	It is positioned for intelligence/cost balance, supports tools including web search, and costs \$2/\$12 per million input/output tokens. Script-generation cost is small relative to media.	GPT-5.6 Sol for difficult, high-stakes review; GPT-5.6 Luna only for low-risk bulk metadata.
Independent claim verification	Google Gemini 3.8 Flash	Use a second provider to check citations and unsupported claims; it supports paid search grounding and is priced at \$0.75/\$3.75 per million input/output tokens through 2026.	GPT-5.6 Terra with a separate verifier prompt when a second provider is unavailable.
Medical educational review	GPT-5.6 Sol plus clinician approval	High-quality model assistance is not a release authority. Medical content must remain source-grounded and clinician-approved.	Hold content; never downgrade to automatic publication.
Core illustrations and consistency edits	Gemini 3.1 Flash Image (Nano Banana 2)	The provider identifies it as the general workhorse for multiple reference images and sequential consistency, which matters more than isolated image beauty.	Gemini 3 Pro Image for premium art direction; regenerate only the failed scene.
Simple high-volume decorative visuals	Gemini 3.1 Flash Lite Image (Nano Banana 2 Lite)	Provider's lowest-cost/fastest option; suitable only when cross-scene reference consistency is not required.	Reuse existing generated assets before issuing another request.

Job	Default	Why	Fallback / escalation
Diagrams, labels, equations, charts, arrows, and captions	Typed SVG + Remotion components	Generated images can misspell labels or depict incorrect scientific relationships. Deterministic code is cheaper, accessible, and editable.	None; do not delegate factual diagrams to an image model.
Long-form teaching narration	ElevenLabs v2 Multilingual	Stable for long-form speech, with one consistent voice and alignment timestamps for caption generation.	ElevenLabs v3 when expressive multi-speaker delivery is required; Flash/Turbo for previews.
Private/offline drafting and safety classification	gpt-oss-20b / gpt-oss-safeguard-20b	Apache 2.0 open weights, controllable deployment, and suitable for drafts, routing, and policy checks.	gpt-oss-120b where 80 GB GPU capacity is justified; use hosted models for final high-stakes review.

Routing and reliability requirements

- Resolve a concrete provider, model identifier, version when available, capability configuration, and fallback policy before a stage starts. Persist the resolved route on the stage checkpoint; do not rely on a mutable provider default for reproducibility.
- The Intake Briefing Agent is a separate pre-run route, resolved as `openai/gpt-5.6-luna` through AI Gateway. It has no retrieval or media tools and may emit only `intake-brief/v1`; its output must be schema-validated before `createVideoRun` is called.
- Persist provider request identifiers, reported token/unit usage, cached and reasoning-token details where available, context-projection version, pricing-version metadata, cost, latency, and outcome for every attempt. Use those records to compare accepted-video cost; never silently trade away evidence or quality to meet a cost target.
- A text-generation route that produces a fact pack, blueprint, script, asset brief, manifest, or QA artifact must support the stage's typed output contract. Validate the returned artifact independently of the provider's structured-output claim.
- Capability gating is mandatory: do not start work that requires unavailable credentials, search, image, TTS, storage, renderer, approval, or model features. Optional enrichment may be omitted only under the process document's recorded fallback rules.
- A fallback may address a transient provider failure only when it can satisfy the same stage contract and quality gate. It must create a new recorded attempt and undergo the same validation and verification; it must never silently downgrade a critical artifact.
- Model routing separates generation from verification where the stage requires independent review. A generator cannot be the sole factual or release authority for its own artifact.

Why this is not a single-model system

- Text models draft and reason, but retrieval plus a separate verifier makes factual failure visible.
- Image models add illustration; they do not author factual diagrams or final text.
- TTS is selected for pronunciation and alignment, not text reasoning.
- Remotion turns approved data into repeatable motion graphics; it is not an AI video model.

This separation makes the system cheaper: the expensive quality model is called a few times per video, images are generated only for genuinely illustrative scenes, and reusable vector assets avoid repeated image calls.

Open-weight position

`gpt-oss-20b` is the practical first open-weight trial: its weights are Apache 2.0 and it can run with 16 GB memory. It is useful for private deployments, routing, first-pass scene plans, and safety-policy classification. It should not be the only factual or medical authority. Self-hosting cost includes GPU time, operations, monitoring, and upgrades, so benchmark total cost per accepted video rather than calling it free.

Source snapshot

- [OpenAI model selection and prices](#)
- [Gemini API prices](#)
- [Gemini image-model selection](#)
- [ElevenLabs API prices](#)
- [Open-weight gpt-oss details](#)

React-to-MP4 rendering and deterministic spatial compositing

Decision

Use React/Remotion as the **deterministic compositor and MP4 renderer**, but do not use Remotion Studio as part of production. A worker can bundle the composition once, pass a locked manifest as input props, and render headlessly with `@remotion/renderer`. This removes the manual Studio adjustment loop; it does **not** remove rendering work. An MP4 still requires a renderer to turn each React frame into pixels and encode/mux audio. The official `renderMedia()` API is designed for this programmatic use and accepts a bundle, a selected composition, JSON input props, and an output path.

This note is an implementation design, not a change to the governing pipeline. It preserves the required order: locked script and visual bible, assets, one full-script voice track, timings/captions, locked timeline, preview QA, approval, final render. In particular, it does not replace the required deterministic Remotion composition with an AI video generator.

What the two reference checkouts do

`reference/template-prompt-to-video` is a mirror of `reference/remotion/packages/template-prompt-to-video` (apart from its Git metadata and `package.json`). The useful mechanism is:

1. The CLI creates story text, one illustration and one TTS clip per story item.
2. `generateVoice()` calls ElevenLabs' timestamped TTS endpoint and writes both the MP3 and character start/end times to `descriptor.json`. ElevenLabs documents this endpoint as returning the audio plus character-level alignment specifically for audio/text synchronisation. [ElevenLabs: Create speech with timing](#)
3. `createTimeLineFromStoryWithDetails()` accumulates each clip duration into absolute `startMs / endMs` values. It converts aligned characters back into short text chunks.
4. `AIVideo.tsx` converts the very same millisecond intervals to frames and mounts a background, caption, and `<Audio>` in `<Sequence>` components. Therefore the spoken clip and its caption start from the same timeline clock. `Root.tsx` derives composition duration from that timeline.
5. A headless `npmx remotion render` (or the `renderer` API) loads the React composition frame by frame and encodes the MP4.

The core lesson is: **audio is the clock**. Captions must be a pure derivation of the alignment of the actual rendered narration, not a separately generated or paraphrased subtitle.

Why the reference template cannot solve precise overlays

Reference behaviour	Consequence
<code>Background.tsx</code> makes every generated image a centred, full-screen, zoomed background.	It stores neither an object's position nor a point such as an eye centre. The same image is also cropped differently while zooming.
<code>TimelineSchema</code> has backgrounds, text and audio only. The text <code>position</code> field is not consumed by <code>Subtitle.tsx</code> .	There is no typed scene graph, z-order, safe area, transform, anchor, or collision rule.
The source image is 1024×1792 while the target is 1080×1920, and the background component starts from a width of the video height (1920).	The fixed cover/crop and 1–1.5× zoom can move a face far from its intended visual location even when the source asset looked correct.
Captions are reconstructed by splitting the source string and walking character indices.	This is fragile for punctuation, repeated whitespace, Unicode, and TTS text normalisation; it also produces very short caption chunks instead of word-level emphasis.
The CLI makes several per-scene TTS requests and <code>getTextAnimations()</code> uses <code>Math.random()</code> .	Per-scene speech conflicts with the required single full-script voice track; unseeded randomness conflicts with reproducible manifest rendering.

The prompt is not the missing coordinate system. Asking an image model for “put object B exactly on object A” can influence a single raster, but it cannot create reusable coordinates shared by independently generated or reused assets. Practitioner reports reach the same practical conclusion: detailed placement instructions drift without spatial conditioning; a control image, depth/line-art guide, or compositing is needed. See the discussion [on spatial consistency](#) and the [layered-composition workflow discussion](#). Those reports are useful experience, not release-quality evidence.

Proposed solution: a constraint-based spatial scene graph

Treat each scene as a small, versioned scene graph in a canonical coordinate space, rather than as a stack of images described in prose.

```
approved base portrait --landmark detection--> face.leftEye / face.rightEye
|
locked camera transform -----+--> deterministic EyeBeam SVG
full voice audio --word alignment--> captions ---+--> same frame clock
```

The LLM may choose *semantic* relationships— `beam originates from character.face.leftEye`, `callout points to circuit.resistor.R1`, or `reaction icon enters above product-card.logo`—but it must not invent raw production `x / y` values. A deterministic layout compiler resolves those relationships after the selected assets are known, then applies the same cover/crop/camera matrix that React uses for every related layer. Thus, if an image pans or zooms, any object attached to it stays attached on every frame.

For the eye-beam example, generate one approved portrait with enough headroom; detect its face landmarks; and render the red beam as a procedural SVG/React effect, not as a second generated bitmap. A beam is a visual effect, so SVG gives exact origin, direction, width, glow, and timing. It is also cheaper and does not risk a mismatched eye-sized PNG. The same contract supports a character holding a generated item, a label attached to a machine part, a sticker on a phone, a foreground cutout behind a desk, particles

emitting from a rocket, or an illustrative image constrained to a caption-safe region. This is compatible with the policy that factual diagrams, labels, equations, charts, and captions are deterministic SVG/Remotion components.

MediaPipe Face Landmarker is a suitable validator/resolver candidate: it returns normalised 3-D face landmarks and transformation matrices intended for effects rendering, including in a Web/JavaScript integration. [MediaPipe Face Landmarker for Web](#) Use it in an asset-validation worker, **not inside the per-frame React render**. Persist its result with the asset hash and detector/model version.

The four coordinate spaces

Every layer must declare which coordinate space it uses. Mixing these spaces is the usual reason an overlay looked correct at one frame and drifted after a crop or camera move.

Space	Unit	Purpose	Example
Asset-local	Normalised (0..1, 0..1) in the uncropped source asset	Persistent detected/authored anchors	<code>face.leftEye</code> , <code>product.logoCentre</code> , object-mask bounds
Layer-local	Normalised within an overlay's own bounds	The point on the overlay that attaches to its target	Beam emitter at (0.05, 0.5) ; speech-bubble tail tip
Scene/world	Canonical 1920×1080 or 1080×1920 design plane	Constraint solving and z-order before camera motion	A cutout is left of a chart by one chart width
Output screen	Integer output pixels at a specific frame	Final React/CSS/SVG render and visual QA	(613, 427) at frame 184

An asset anchor never becomes a screen pixel directly. The layout compiler maps it through asset fitting, its parent group transform, and the scene camera in that order. Captions and other screen UI live in a `screen-ui` group which the camera does not transform.

Layer types and anchor providers

The asset record must specify usable geometry, not merely its file URL.

Layer type	Required geometry	Typical placement use
Background image	Source dimensions and <code>cover / contain</code> crop transform	Full scene context under camera motion
Subject cutout	Verified alpha or segmentation mask, visible bounds, subject anchors	Place character/product in front of another layer without rectangular edges
Generated illustrative image	Source dimensions, safe-region result, optional detected object boxes/keypoints	A self-contained scene element; do not use it as a precise attachment target unless validation finds its target anchor
Procedural SVG/effect	Local attachment point(s), scalable dimensions, optional path	Beams, arrows, glows, highlights, callouts, particles, masks
Deterministic diagram/text/caption	Typed component bounds and named ports/labels	Exact factual visuals and caption-safe layout

Anchor providers are ordered by reliability: authored vector port or SVG ID; deterministic component bounds; validated alpha-mask boundary; specialised vision landmarks (face/hand/pose); validated object detection; and finally an explicitly reviewed manual anchor. A prose description or an LLM-proposed pixel coordinate is never an anchor provider.

A generated image's dimensions are not its placement contract

The LLM should request an aspect ratio, enough resolution for the largest displayed size, visible-subject/safe-region requirements, and required anchors. It should **not** attempt to make two independently generated assets “the same size” so that they will happen to align. Source pixels are only sampling quality; the resolved display size comes from the target's measured geometry at render time.

An independently generated overlay must be delivered as one of the following:

- an RGBA cutout with a verified alpha boundary and at least one local anchor;
- a source image plus a persisted segmentation mask and visible bounds; or
- a self-contained illustration that is not expected to align to another asset.

For example, an attached helmet can use its mask's bottom-centre as `localAnchor`, then attach to a detected head-top/face anchor and scale from the head width. A generated flat rectangular "helmet image" with no mask, no anchor, and a human-written `left/top` is an invalid attachment asset. Effects, arrows, glows, labels, brackets, lines, and highlights should normally be deterministic components rather than generated overlays.

Two locked artifacts, not one vague timeline

Keep both artifacts below. The second is derived deterministically and is the only geometry React consumes.

1. `scene-plan/v1` : LLM-produced semantic intent such as `left-of`, `attach`, `behind-mask`, `safe area`, required subject/anchor names, visual purpose, asset-role briefs, and timing references to narration words.
2. `resolved-layout/v1` : compiler output after the selected asset hashes, source dimensions, masks, landmark/detection results, canvas, and camera keyframes are locked. It holds the matrix/bounds/clip-path for each sampled placement interval and records every constraint decision.

The final video manifest references hashes for both artifacts. A replacement image invalidates and recompiles `resolved-layout/v1`; it cannot reuse geometry from a different candidate image.

Manifest shape

This abbreviated TypeScript-like manifest shows the required separation of semantic intent, detected anchors, and deterministic display transforms.

```

type Point = {x: number; y: number}; // normalised [0, 1] in source-image space

type Asset = {
  id: string;
  sha256: string;
  mimeType: "image/png" | "image/webp" | "audio/wav";
  sourceWidth?: number;
  sourceHeight?: number;
  provenance: {role: "illustration" | "voice"; promptVersion?: string};
  anchors?: Record<"face.leftEye" | "face.rightEye", Point>;
};

type Layer = {
  id: string;
  kind: "image" | "cutout" | "svg-effect" | "diagram" | "caption";
  assetId?: string;
  zIndex: number;
  startMs: number;
  endMs: number;
  parentId: "world" | "camera" | "screen-ui" | string;
  placement: Placement;
  occlusion?: {mode: "front" | "behind-mask" | "clip-to-mask"; targetLayerId?: string};
  effect?: {type: "eye-beam"; lengthEyeWidths: number; widthEyeWidths: number};
};

type Placement =
  | {kind: "fit"; fit: "cover" | "contain"; focalAnchor?: string}
  | {kind: "absolute"; origin: Point; size: {w: number; h: number}; reviewId: string}
  | {
    kind: "attach";
    target: {layerId: string; anchor: string};
    localAnchor: Point;
    offset: {x: number; y: number; unit: "target-width" | "target-height" | "anchor-distance";
    size: {value: number; basis: "target-width" | "target-height" | "anchor-distance";
    rotation: "inherit-target" | "outward" | {degrees: number};
  }
  | {
    kind: "relative";
    relation: "left-of" | "right-of" | "above" | "below" | "centered-in";
    targetLayerId: string;
    gap: {value: number; unit: "target-width" | "target-height";
  };

type Word = {text: string; startMs: number; endMs: number};
type VideoManifestV1 = {
  schemaVersion: "video-manifest/v1";
  fps: 30;
  canvas: {width: 1080; height: 1920; safeArea: {top: 120; right: 72; bottom: 280; left: 72};
  narration: {assetId: string; words: Word[]; alignmentSource: "tts" | "forced-alignment";

```

```
scenes: Array<{id: string; claimIds: string[]; layers: Layer[]}>;
};
```

`absolute` placement is deliberately exceptional: it must name a review/author decision and is useful only for brand locks or composition templates. Normal scene planning uses `attach` and `relative` placement. `CameraKeyframe` must be fully specified and seeded; it contains pan, zoom and rotation in time, not an ad-hoc CSS adjustment. The resolver records the resolved pixel anchor at each camera keyframe or resolves it mathematically at render time from the same keyframes.

Constraint compilation and collision policy

The scene planner produces a graph of relationships; a deterministic compiler then resolves it topologically. It first resolves base asset fit/crop, then intrinsic anchors, relative placements, attachment transforms, parent camera transforms, and finally screen-ui. It must fail on an anchor cycle, a missing target, incompatible coordinate space, out-of-bounds required anchor, or unsatisfiable safe-area/collision constraint.

Each layer also declares a bounding box (or alpha-mask bounds) and one of these policies:

- `must-not-overlap` : captions, brand marks, labels, or independent subjects;
- `may-overlap` : decorative layers where z-index decides the appearance;
- `attach-overlap` : an effect deliberately overlaps its target at its anchor;
- `behind-mask` / `clip-to-mask` : the target's validated mask creates true foreground/background occlusion.

Collision is checked at all placement keyframes and sampled intermediate frames, not only at frame zero. The compiler may use deterministic alternatives stated in the visual bible (move a callout to another named side, reduce it within a bounded range, or pick another approved scene layout). It must never make a silent, arbitrary move.

Generic example: several overlays, not a face-specific special case

```
Scene world
├ lab-photo          fit: cover, focalAnchor: microscope.stage
├ specimen-cutout    relative: right-of(lab-photo.subjectBounds), gap: 0.06 target-width
├ magnifier-svg      attach: specimen-cutout.subject.centre, localAnchor: lens.centre
├ pointer-arrow-svg  attach: lab-photo.microscope.stage, localAnchor: arrow.tip
├ annotation-card    relative: below(specimen-cutout), must-not-overlap: captions
└ captions           parent: screen-ui, constrained to bottom safe area
```

If the camera pushes into `lab-photo`, the arrow follows the microscope stage, because both use the camera parent. The specimen and annotation remain in the world layout unless their parent says otherwise. Captions remain legible at the screen safe area because they are deliberately outside the camera group. This is the same system that places a human, eye beam, product, logo, callout, mask, or any number of arbitrary layers.

Coordinate math that prevents drift

Let source dimensions be (S_w, S_h) and canvas dimensions be (C_w, C_h) . For a `cover` image, first calculate:

```
s = max(Cw / Sw, Ch / Sh)
drawW = Sw * s      drawH = Sh * s
offsetX = (Cw - drawW) / 2    offsetY = (Ch - drawH) / 2
```

An asset landmark (u, v) is normalised in its *uncropped source image*. Its canvas point before camera motion is:

```
p = (offsetX + u * drawW, offsetY + v * drawH)
```

Apply the exact same affine camera transform $M(t)$ to both the image and p . An attached overlay's transform is $M(t) \times \text{parentTransform} \times \text{targetAnchor} \times \text{attachmentOffset} \times \text{inverse}(\text{localAnchor})$. The beam's SVG is then translated to the resulting point, scaled from the eye width, and rotated according to the chosen outward eye direction. Do not put independent `left`, `top`, `scale`, or `zoom` values on an attached overlay. They will eventually drift.

When there are no reliable anchors (profile face, occlusion, multiple faces, or detector confidence below the declared threshold), the asset fails the scene contract. Regenerate the **base asset**, use a valid preapproved alternative, switch to an approved self-contained layout, or require review; never silently guess a nearby coordinate.

Asset brief and prompt contract

The LLM should create an image brief, not a final layout guess. Example:

```
{
  "role": "character-base-portrait",
  "canvas": {"aspectRatio": "9:16", "safeRegion": [0.08, 0.10, 0.92, 0.72]},
  "requiredAnchors": ["face.leftEye", "face.rightEye"],
  "acceptance": [
    "exactly one unobstructed frontal face",
    "both eyes visible and inside the safe region",
    "head and shoulders; leave clean space to the subject's left",
    "no text, labels, glasses glare, eye effects, or crop through the face"
  ],
  "styleRefIds": ["locked-visual-bible-character-sheet"]
}
```

If the image provider supports a layout/reference image, create a deterministic SVG guide with the safe region and simple face/eye placeholders, and use it as spatial conditioning or an edit input. This can improve the candidate rate, but it is still only an aid: detector validation and deterministic compositing remain the authority. Community examples similarly use depth maps or collage layers as structural guidance rather than trusting prose placement. [NVIDIA/ComfyUI depth-map example](#)

Captions and audio: one clock, quantised once

1. Generate the complete approved narration once. Persist audio bytes, hash, actual measured duration, TTS model/voice/settings, and the provider's alignment response.
2. Convert character alignment to word tokens by offsets in the exact text sent to TTS. Preserve punctuation and whitespace offsets; do not rediscover words using `split(" ")`.
3. Independently run forced alignment against the saved audio and locked script when the provider alignment is unavailable or as the QA verifier. ElevenLabs' forced-alignment API returns word and character timestamps plus an alignment loss value. [ElevenLabs: Forced Alignment](#)
4. Create readable phrase cues from consecutive words while retaining each word's own bounds for highlighting. The caption string must reconstruct the approved narration exactly.
5. Convert milliseconds to frames through one shared function, for example `startFrame = floor(startMs * fps / 1000)` and `endFrameExclusive = ceil(endMs * fps / 1000)`. A sequence duration is `max(1, endFrameExclusive - startFrame)`. Store the original milliseconds as evidence; use these quantised frame bounds everywhere in React.
6. Mount the one narration `<Audio>` at frame zero. Caption and visual beats independently refer to its absolute word/cue intervals. A frame at 30 fps is 33.33 ms, well inside the existing 150-ms p95 word-alignment gate, while the separate alignment QA detects larger errors.

Remotion also provides caption utilities for importing, displaying, and exporting timed captions; its SRT parser represents captions with explicit `startMs` and `endMs`. [Remotion captions documentation](#)

Headless React-to-MP4 workflow

```
locked manifest + verified local/object-store assets
-> cached composition bundle (only when code/dependencies change)
-> selectComposition(bundle, inputProps: manifest)
-> renderMedia(... codec: "h264", outputLocation: immutable MP4 path)
-> ffprobe/render QA + immutable release record
```

The render worker should expose a job command/API, not Studio:

```
const serveUrl = await bundle({entryPoint: "src/index.ts"}); // cache by lockfile + commit
const composition = await selectComposition({serveUrl, id: "Lesson", inputProps: manifest});
await renderMedia({
  serveUrl,
  composition,
  codec: "h264",
  audioCodec: "aac",
  inputProps: manifest,
  outputLocation: finalMp4Path,
});
```

This uses headless Chromium and FFmpeg under the renderer; that compute is necessary for an MP4, but Studio is not started and no human needs to drag an overlay. Cache the bundle, constrain renderer concurrency/memory, and render a small set of deterministic validation frames before the full preview. Never fetch an unverified provider URL inside the composition—use validated asset references resolved from the manifest.

Required QA before adopting this design

These are proposed implementation checks that supplement, rather than change, the existing governing release gates.

Check	Deterministic evidence	Failure handling
Manifest/asset integrity	Zod/JSON-schema validation; asset hash, MIME, dimensions, provenance, detector and model version present	Block scene assembly.
Anchor availability	Exactly one required face and both named eyes detected in source coordinates; source aspect/crop contract passes	Regenerate or replace only the illustration asset.
Anchor attachment	At start, middle, end, and every camera keyframe, the beam origin equals the transformed eye anchor within 1 output pixel	Block preview; fix the renderer/manifest transform.
Effect scale and direction	Beam width/length are derived from detected interpupillary distance and pass declared min/max bounds	Block preview or require review.
Caption correctness	Reconstructed words exactly equal the locked narration; monotonic non-overlapping cue bounds; safe-area and contrast checks	Rebuild caption derivation.
Audio/caption sync	Independent forced alignment; p95 word error remains within the governing 150-ms limit	Regenerate/re-align voice, then rerun dependent work.
Render integrity	Preview/final frame count, measured audio/video duration, no missing assets, and sampled anchor/caption screenshots	Block release.

Add regression fixtures for: a frontal face with a 2× zoom/pan, portrait versus landscape source images, a rejected profile face, Unicode/punctuation caption alignment, a TTS-normalised number, a missing asset, and a recovered rendering job. Record all output hashes, timing/QA results, model versions, prompt template version, detector version, cost and latency in the run checkpoint.

Practical implementation order

1. Replace the template's three-list timeline with a versioned manifest and a fixed 9:16 coordinate system; remove unseeded randomness.
2. Implement a single full-script narration/alignment artifact and a pure caption derivation module with tests.
3. Add `ImageLayer`, `CutoutLayer`, `ConstraintLayout`, and `AnchorEffectLayer` React components. They must consume `resolved-layout/v1` and share one source-to-canvas transform function; implement one procedural effect (such as `EyeBeam`) only as a regression example, not as the architecture's focus.
4. Add the off-render asset validation worker and persist landmark results with selected asset provenance.
5. Build headless preview/final render jobs plus the QA checks above. Only then trial controlled image-generation inputs (guide/reference images) to improve asset acceptance rate.

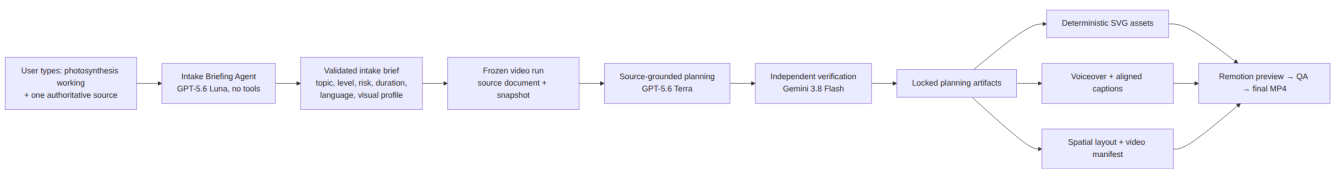
This order attacks the actual cause of manual `x / y` adjustment: coordinates and attachments become data verified from the approved image, not an LLM's best-effort prose instruction.

Source-grounded lesson planning and structured scene manifests

This is the current, implementation-backed explanation of the **Source-grounded lesson planning and structured scene manifests** job. It now receives a complete, typed brief from the Chat intake path rather than asking the user to fill the long form first.

It does not create an MP4 by itself. Its job is to turn a frozen lesson brief and supplied evidence into verified planning artifacts that the asset, voice, layout, manifest, render, and QA stages can safely consume.

The whole handoff at a glance



The two text-model responsibilities are intentionally separate:

Job	Model and tools	Creates	Must not create
Intake Briefing Agent	GPT-5.6 Luna through AI Gateway; no tools	A compact intake-brief/v1 metadata object	Research, citations, claims, narration, visual assets, or medical advice
Source-grounded lesson planning	GPT-5.6 Terra using only locked source-derived context	Fact pack, lesson blueprint, approved script, and visual bible	New unsupported facts, a final video, or unverified diagrams
Independent verification	Gemini 3.8 Flash	Fact-support and script-support decisions	A replacement lesson plan or rewritten narration

What the chat path gives this job

For a request like `photosynthesis working`, Chat requires one supplied source as either:

- pasted text after a `Source:` line;
- an HTTPS URL; or
- an uploaded TXT, Markdown, or text-based PDF.

The upload path extracts text deterministically, limits the file to 10 MiB and the extracted text to 100,000 characters, hashes both raw bytes and retained text, and stores file bytes privately. Chat does not search the web or invent a source.

Before Luna is called, the system creates a durable `intake_sessions` record. The agent receives only the request text and selected language. Its strict output is validated against `intake-brief/v1`:

```
{
  "schemaVersion": "intake-brief/v1",
  "topic": "How photosynthesis works",
  "learningLevel": "Grade 8",
  "domain": "standard",
  "audienceCategory": "school",
  "durationSeconds": 60,
  "language": "en",
  "visualProfile": "Precise, calm educational motion graphics"
}
```

This is only a metadata brief. It contains no source text, claim, citation, scene, narration, timing, or image prompt. The system records the request hash, brief hash, model, prompt version, token counts when available, latency, and each attempt. A malformed or unavailable briefing fails visibly and does not create a video run.

Once valid, the service combines the brief with fixed run defaults— 16:9 and local destination—and the user source. It calls `createVideoRun`, which freezes the following `input-snapshot/v1` shape:

```
{
  "schemaVersion": "input-snapshot/v1",
  "topic": "How photosynthesis works",
  "learningLevel": "Grade 8",
  "audienceCategory": "school",
  "language": "en",
  "durationSeconds": 60,
  "aspectRatio": "16:9",
  "domain": "standard",
  "visualProfile": "Precise, calm educational motion graphics",
  "requestedDestination": "local",
  "sourceIds": ["<source-document-uuid>"]
}
```

Changing any of these values or the source creates a different run; it does not mutate an existing lesson.

What the source-grounded planning job creates

The following outputs are stored as versioned JSON artifacts tied to one `video_runs` record. They are database records, not Markdown files generated beside the project. Every persisted artifact includes a schema version, input hash, content hash, validation status, provenance, and timestamps.

Order	Stage	Artifact created	What it contains	What receives it next
1	Research	<code>source-evidence-map / source-evidence-map/v1</code>	Every retained source split into stable, hashed text segments with offsets	Fact-pack generation and fact verification
2	Research	<code>fact-pack / fact-pack/v2</code>	Bounded claims, caveats, and exact source/segment references for each supported claim	Independent fact verification, blueprint, script

Order	Stage	Artifact created	What it contains	What receives it next
3	Fact verification	fact-verification / fact-verification/v2	One independent supported/not-supported result for every fact-pack claim	Claim records, release evidence, planning gate
4	Blueprint	lesson-blueprint / lesson-blueprint/v1	Learning objective, prerequisites, ordered scene IDs, scene purposes, claim IDs, and visual beats	Script and manifest assembly
5	Script	approved-script / approved-script/v2	Canonical narration lines, each tied to a scene ID, claim IDs, and a visual action	Voiceover, visual bible, assets, layout, manifest
6	Script verification	Validation result is checked inline; provider usage is recorded	Each narration line must be supported by the claims it names	Gate before the approved script is persisted
7	Visual bible	visual-bible / visual-bible/v1	Palette, typography, caption safe area, persistent entities, camera behavior, and forbidden visual patterns	Deterministic SVG assets, layout, captions, manifest

The fact verifier is a stored artifact. Script verification currently runs and blocks unsafe narration, but the code does **not yet persist** a separate `script-verification` artifact; it records the provider attempt and only persists the approved script after the check passes. That is an intentional current-state distinction, not a claim of a missing artifact being present.

Worked example: what moves through the planner

The objects below illustrate structure and links only. They are not approved photosynthesis content unless the user-provided source supports them.

1. Source evidence map

The planner never receives a free-form web search result. Research first locks the source into deterministic segments:

```
{
  "schemaVersion": "source-evidence-map/v1",
  "sources": [
    {
      "sourceId": "<source-document-uuid>",
      "sourceHash": "<64-character-content-hash>",
      "segments": [
        {
          "id": "<stable-segment-id>",
          "ordinal": 0,
          "startOffset": 0,
          "endOffset": 184,
          "text": "<the retained user-supplied source text>"
        }
      ]
    }
  ]
}
```

2. Fact pack

Terra receives the segmented source and may output only claims linked to those segments:

```
{
  "schemaVersion": "fact-pack/v2",
  "claims": [
    {
      "id": "<claim-uuid>",
      "text": "<a claim supported by the supplied source>",
      "evidence": {
        "sourceId": "<source-document-uuid>",
        "sourceHash": "<64-character-content-hash>",
        "segmentIds": ["<stable-segment-id>"],
        "locator": "supplied source, segment 1"
      },
      "critical": true
    }
  ],
  "caveats": []
}
```

Gemini independently checks every proposed claim against only its cited source segments. If one is unsupported, the run fails at this point; it does not continue with a plausible-sounding explanation.

3. Lesson blueprint — the first structured scene plan

The blueprint converts verified facts into an educational sequence. It is the first scene-oriented artifact, but it is not yet the renderer manifest.

```
{
  "schemaVersion": "lesson-blueprint/v1",
  "objective": "Explain the supported inputs, transformation, and outputs of photosynthesis.",
  "prerequisites": ["Plants use leaves"],
  "scenes": [
    {
      "id": "<scene-1-uuid>",
      "order": 0,
      "purpose": "Introduce the learner question.",
      "claimIds": [],
      "visualBeat": "A leaf appears and a question becomes a simple process diagram."
    },
    {
      "id": "<scene-2-uuid>",
      "order": 1,
      "purpose": "Explain one supported part of the process.",
      "claimIds": ["<claim-uuid>"],
      "visualBeat": "Show an input arrow moving into the persistent leaf."
    }
  ]
}
```

It forwards stable scene IDs, ordering, claimed evidence, and intended canvas changes. It does **not** include pixel coordinates, asset IDs, captions, word timings, or MP4 timings.

4. Approved script

The script stage receives a minimal projection of the blueprint and the claims needed by each scene. It produces the one canonical narration representation:

```
{
  "schemaVersion": "approved-script/v2",
  "narration": [
    {
      "id": "<line-uuid>",
      "sceneId": "<scene-2-uuid>",
      "text": "<a source-supported, learner-appropriate explanation>",
      "claimIds": ["<claim-uuid>"],
      "visualAction": "Draw the supported process element beside the persistent leaf."
    }
  ]
}
```

Gemini verifies every line before this script is saved. Later stages derive the complete TTS text deterministically from these lines, so there is no second, drifting narration copy.

5. Visual bible

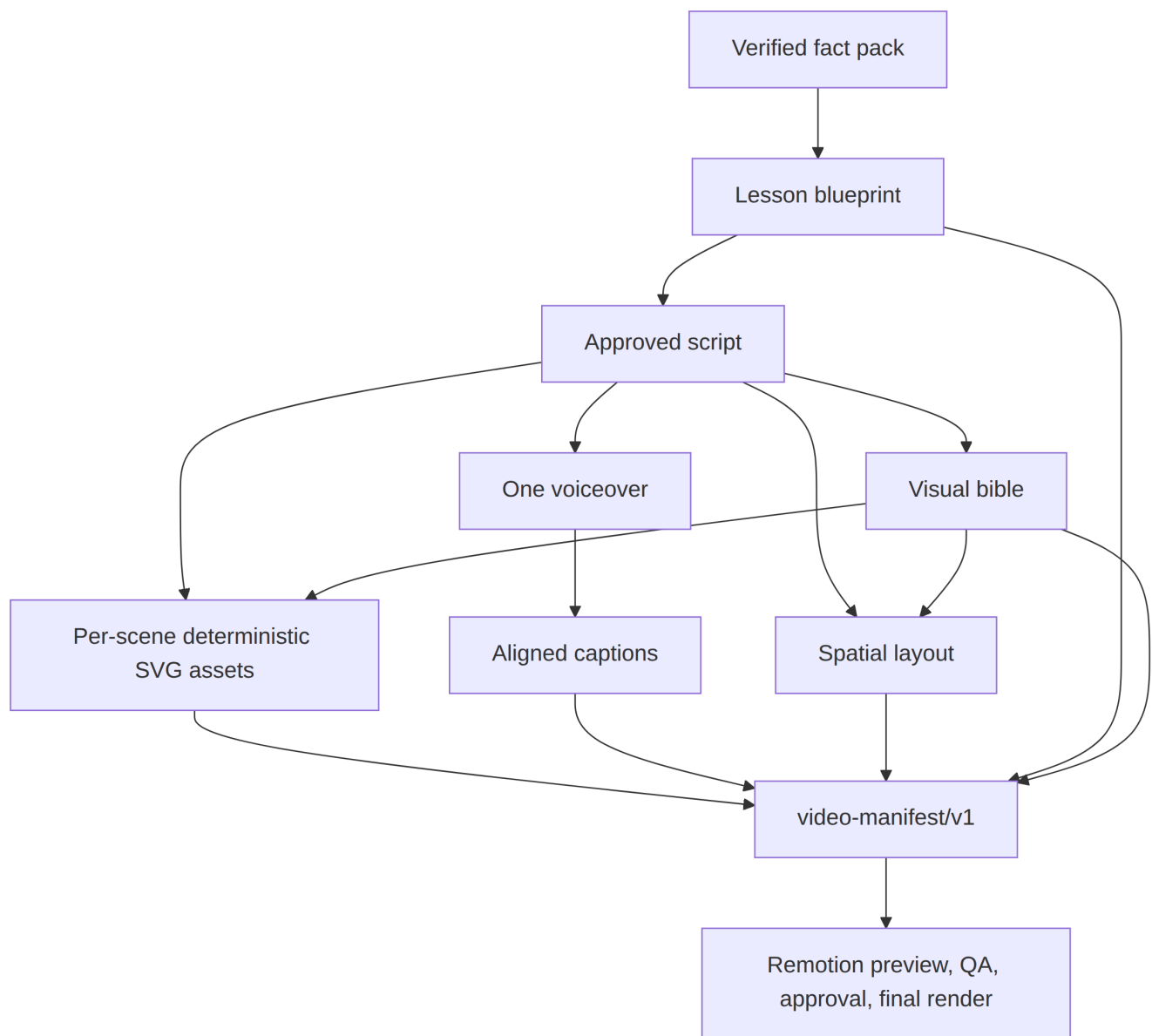
The visual bible locks the visual language before anything is rendered:

```

{
  "schemaVersion": "visual-bible/v1",
  "canvasTexture": "warm off-white paper",
  "lineStyle": "rounded dark-green marker strokes",
  "palette": ["#155E3A", "#F2C94C", "#5DADE2"],
  "typography": {
    "heading": "bold rounded sans serif",
    "body": "clear sans serif",
    "caption": "high-contrast sans serif"
  },
  "captionSafeArea": {"top": 0.08, "right": 0.07, "bottom": 0.12, "left": 0.07},
  "persistentEntities": [
    {"id": "leaf-01", "description": "The same leaf remains visible across related scenes."}
  ],
  "camera": {"behavior": "one evolving canvas", "transitions": ["draw-on", "gentle pan"]},
  "prohibitedVisualPatterns": ["text baked into generated illustrations"]
}

```

What is forwarded after planning



Downstream receiver	Planning inputs it consumes	New output it creates
Asset stage	Approved script + visual bible	One selected private SVG asset for each narrated scene. The current implementation is deterministic SVG and omits optional generated illustrations.
Voiceover	Canonical approved-script narration	One MP3 plus provider word alignment.
Captions	Voice word alignment	Caption cues tied to exact aligned word indexes.
Spatial layout	Approved script + run aspect ratio	Resolved canvas and layer bounds per scene.
Manifest assembly	Blueprint, script, visual bible, selected assets, layout, voiceover, captions	video-manifest/v1 : scene start/end times, layer asset IDs, caption cues, safe area, and narration asset ID.
Remotion/render QA	Locked video manifest and verified private assets	Preview MP4, QA evidence, approval decision, final MP4, and a release record.

The precise scene timings are not guessed by the planner. They are assembled after TTS returns word alignment. The final `video-manifest/v1` is the renderer contract; the blueprint is the educational-scene contract that comes before it.

Current-state boundaries

This document distinguishes what the repository does today from what its contracts may support later:

- `scene-plan/v1` and `scene-asset-brief/v1` schemas exist, but current stages do not generate or persist those artifacts yet.
- The current asset stage creates deterministic SVG scene assets. They are reliable renderer inputs, but they are not yet a complete semantic, labelled photosynthesis diagram component.
- Factual diagrams, labels, equations, charts, arrows, and captions must remain typed SVG/Remotion work. They must not be delegated to image generation.
- The worker currently advances stage jobs in its declared order. Per-scene SVG creation is concurrent inside the asset stage; whole downstream stages are not yet independently dispatched in parallel.
- Medical requests are classified conservatively at intake and remain educational-only; clinician approval is still required before medical publication.

Records created for audit and reproduction

Record location	Why it exists
<code>intake_sessions</code> , <code>intake_attempts</code>	Durable chat request, validated brief, hashes, Luna route, usage, latency, and retry outcome
<code>video_runs</code> , <code>source_documents</code>	Frozen snapshot, source identity, private object key when uploaded, text/raw-byte hashes, and retrieval metadata
<code>artifacts</code> , <code>artifact_attempts</code> , <code>stage_checkpoints</code>	Versioned planning outputs, model/provider route, validation, retries, and locked dependency hashes
<code>source_claims</code>	Verified claim-to-source links used by the final release record
<code>provider_usage</code>	Request identifiers, model, token/character accounting, cost metadata, latency, and outcome
<code>media_assets</code> , <code>render_outputs</code>	Selected SVG/audio assets and final render bytes with provenance and hashes
Release record artifact	Reproducible run snapshot, intake provenance, source links, artifacts, QA, approval, usage, and output hashes

Implementation map

- [Chat intake API](#) — source parsing/upload handling and durable session creation.
- [Intake orchestrator](#) — Luna call, validated handoff, attempts, retries, and video-run creation.
- [Pipeline stages](#) — research, fact verification, blueprint, script, visual bible, assets, voice, captions, layout, manifest, QA, and release record.
- [Typed contracts](#) — exact schemas for intake, planning, layout, and renderer inputs.

- [Database schema](#) — durable records and provenance fields.
- [Required process](#) — governing dependency order and release policy.